

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 14 – SORTING



NAMA : MOH. CHANDRA WARDANA

NIM : 109082530025

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Pengurutan data atau sorting merupakan salah satu proses fundamental dalam pemrograman yang bertujuan untuk menyusun sekumpulan data ke dalam aturan tertentu, baik secara membesar (ascending) maupun mengecil (descending). Di dalam modul praktikum ini, pembahasan berfokus pada dua metode pengurutan linier yang sederhana namun efektif untuk jumlah data tertentu, yaitu Selection Sort dan Insertion Sort. Kedua algoritma ini bekerja dengan memanfaatkan struktur perulangan bersarang (nested loop) di mana data dimanipulasi di dalam sebuah array atau slice agar berpindah ke posisi indeks yang seharusnya secara bertahap.

Selection Sort adalah algoritma pengurutan yang didasari oleh ide pencarian nilai ekstrim (nilai terkecil atau terbesar) pada sekumpulan data yang belum terurut. Pada setiap tahap perulangan, algoritma ini memindai rentang data yang tersisa untuk menemukan indeks dari nilai ekstrim tersebut, kemudian menukar posisi elemen ekstrim tersebut dengan elemen yang berada di ujung paling kiri dari rentang data tersisa. Proses pencarian indeks dan pertukaran tempat (swap) ini terus diulangi secara linier hingga rentang data tersisa hanya menyisakan satu elemen saja, yang menandakan seluruh isi array telah berhasil disusun dengan benar.

Di sisi lain, Insertion Sort bekerja dengan prinsip yang berbeda, yaitu menyisipkan suatu nilai secara langsung pada posisi yang tepat tanpa harus melakukan pencarian nilai ekstrim terlebih dahulu. Algoritma ini membagi data menjadi dua bagian, yakni bagian yang sudah terurut di sebelah kiri dan bagian yang belum terurut di sebelah kanan. Melalui kombinasi proses pencarian sekuensial (sequential search) dan pergeseran nilai, satu per satu elemen dari bagian yang belum terurut akan diambil, lalu elemen-elemen di bagian kiri yang nilainya tidak sesuai akan digeser ke kanan guna membuka ruang kosong agar elemen baru tersebut dapat disisipkan pada posisi yang tetap menjaga keterurutan data

B. Guided

1. Insertion Sort (Integer - Descending)

```
1 package main
2
3 import "fmt"
4
5 const nMax int = 4321
6
7 type arrInt [nMax]int
8
9 func insertionSort(T *arrInt, n int) {
10     /* I.S. terdefinisi array T yang berisi n bilangan bulat
11        F.S. array T terurut secara mengecil atau descending dengan INSERTION SORT */
12
13     var temp, i, j int
14
15     i = 1
16     for i <= n-1 {
17         j = i
18         temp = T[j]
19
20         for j > 0 && temp > T[j-1] {
21             T[j] = T[j-1]
22             j = j - 1
23         }
24
25         T[j] = temp
26         i = i + 1
27     }
28 }
29
30 func main() {
31     var data arrInt
32     var n int
33
34     fmt.Scan(&n)
35
36     for i := 0; i < n; i++ {
37         fmt.Scan(&data[i])
38     }
39
40     insertionSort(&data, n)
41
42     for i := 0; i < n; i++ {
43         fmt.Print(data[i], " ")
44     }
45 }
46
```

Penjelasan Code :

Program ini mengimplementasikan algoritma *Insertion Sort* untuk mengurutkan kumpulan bilangan bulat secara mengecil (*descending*) berdasarkan struktur data *array* statis berkapasitas maksimal 4321 elemen. Di dalam fungsi `insertionSort`, algoritma bekerja dengan membagi elemen

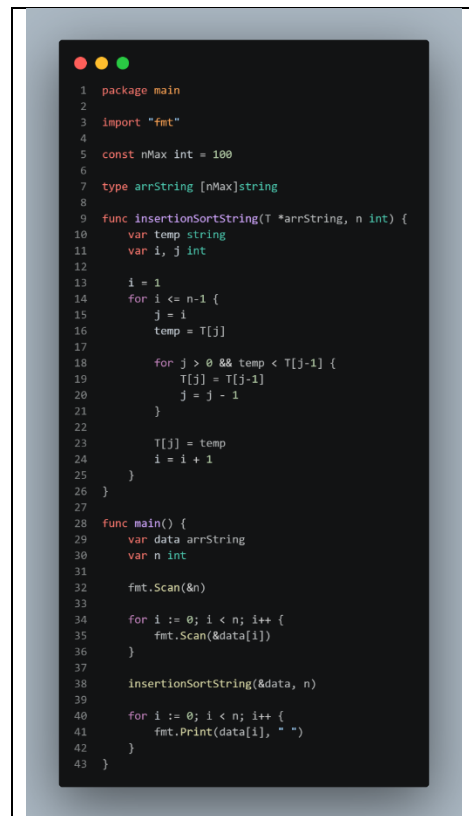
menjadi bagian terurut dan belum terurut, lalu mengambil elemen target (temp) secara bertahap mulai dari indeks kedua ($i = 1$) untuk disisipkan ke posisi kiri yang sesuai melalui proses pergeseran linear selama kondisi $\text{temp} > T[j-1]$ terpenuhi. Pada fungsi utama main, program membaca jumlah elemen n beserta seluruh datanya dari piranti masukan, mengeksekusi fungsi pengurutan dengan mengirimkan alamat memori *array* (&data), dan mencetak isi elemen yang kini telah tersusun dari nilai terbesar hingga terkecil ke layar dengan pemisah spasi.

Output :



```
PS D:\109082530025_MOH-Chandra-Wardana_> go run "d:\109082530025_MOH-Chandra-Wardana_\Week 12 - Modul 14\Guided\guided1\guided1.go"
5
12 45 7 23 19
45 23 19 12 7
PS D:\109082530025_MOH-Chandra-Wardana_>
```

2. Insertion Sort (String - Ascending)



```
1 package main
2
3 import "fmt"
4
5 const nMax int = 100
6
7 type arrString [nMax]string
8
9 func insertionSortString(T *arrString, n int) {
10     var temp string
11     var i, j int
12
13     i = 1
14     for i <= n-1 {
15         j = i
16         temp = T[j]
17
18         for j > 0 && temp < T[j-1] {
19             T[j] = T[j-1]
20             j = j - 1
21         }
22
23         T[j] = temp
24         i = i + 1
25     }
26 }
27
28 func main() {
29     var data arrString
30     var n int
31
32     fmt.Scan(&n)
33
34     for i := 0; i < n; i++ {
35         fmt.Scan(&data[i])
36     }
37
38     insertionSortString(&data, n)
39
40     for i := 0; i < n; i++ {
41         fmt.Print(data[i], " ")
42     }
43 }
```

Penjelasan Code :

Program ini merupakan modifikasi dari algoritma *Insertion Sort* yang

dirancang khusus untuk mengurutkan data bertipe teks (*string*) secara membesar (*ascending*) alfabetis menggunakan batas tampung *array* sebanyak 100 elemen. Alur kerja pada fungsi `insertionSortString` memiliki struktur logika yang serupa dengan pengurutan numerik, namun kondisi perulangan dalamnya diubah menjadi `temp < T[j-1]` agar karakter atau kata dengan urutan leksikografis lebih awal digeser ke arah kiri. Fungsi `main` mengendalikan jalannya program dengan meminta input kuantitas data *n*, mengisi *array* data melalui perulangan `fmt.Scan`, melakukan pemanggilan fungsi urut secara *pass-by-reference*, dan diakhiri dengan mencetak string-string tersebut yang telah terurut rapi mulai dari alfabet A hingga Z.

Output :

```
PS D:\109082530025_MOH-Chandra-Wardana> go run "d:\109082530025_MOH-Chandra-Wardana_
\Week 12 - Modul 14\Guided\guided2\tempCodeRunnerFile.go"
4
Gado-Gado Bakso Soto Ayam
Ayam Bakso Gado-Gado Soto
PS D:\109082530025_MOH-Chandra-Wardana>
```

3. Selection Sort (Integer Array - Ascending)

```
1 package main
2
3 import "fmt"
4
5 func selectionSortArray(angka *[5]int){
6     var idx_min, i, j int
7     for i = 0; i < len(angka) - 1; i++ { //loop luar (iterasi sortingnya)
8         idx_min = i
9         for j = i + 1; j < len(angka); j++ { //loop dalam (mencari nilai ekstrim)
10             if angka[j] <= angka[idx_min] { //sorting terkecil ke terbesar
11                 idx_min = j
12             }
13         }
14         if idx_min != i {
15             angka[i], angka[idx_min] = angka[idx_min], angka[i]
16         }
17     }
18 }
19
20 func main(){
21     var arrAngka [5]int
22
23     for i := 0; i < len(arrAngka); i++ {
24         fmt.Printf("Masukkan data angka ke-%d : ", i)
25         fmt.Scan(&arrAngka[i])
26     }
27     fmt.Println()
28
29     fmt.Println("=== SEBELUM DISORTING ===")
30     for i := 0; i < len(arrAngka); i++ {
31         fmt.Print(arrAngka[i], " - ")
32     }
33
34     fmt.Println("\n=== SETELAH DISORTING ===")
35     selectionSortArray(&arrAngka)
36     for i := 0; i < len(arrAngka); i++ {
37         fmt.Print(arrAngka[i], " - ")
38     }
39 }
```

Penjelasan Code :

Program ini menerapkan algoritma *Selection Sort* untuk menyusun lima buah elemen bilangan bulat di dalam *array* terikat secara membesar (*ascending*) menggunakan teknik perulangan modern di Go. Fungsi `selectionSortArray`

bekerja secara efisien dengan memanfaatkan struktur *looping* luar untuk menetapkan posisi target, diikuti *looping* dalam untuk mencari nilai terkecil (`idx_min`) dari elemen yang tersisa lewat kondisi `angka[j] <= angka[idx_min]`, lalu menukar kedua elemen tersebut secara instan menggunakan fitur *multiple assignment* khas Go jika ditemukan indeks yang tidak sesuai. Bagian fungsi main mendeklarasikan array berukuran tetap `[5]int`, memandu pengguna mengisi data satu per satu, menampilkan kondisi urutan elemen sebelum dilakukan penyaringan, memproses pengurutan, dan memunculkan hasil akhir data terurut ke layar menggunakan format pemisah tanda hubung (*strip*).

Output :

```
PS D:\109082530025_MOH-Chandra-Wardana_> go run "d:\109082530025_MOH-Chandra-Wardana_
\Week 12 - Modul 14\Guided\guided3\tempCodeRunnerFile.go"
Masukkan data angka ke-0 : 89
Masukkan data angka ke-1 : 12
Masukkan data angka ke-2 : 54
Masukkan data angka ke-3 : 23
Masukkan data angka ke-4 : 7

=== SEBELUM DISORTING ===
89 - 12 - 54 - 23 - 7 -
=== SETELAH DISORTING ===
7 - 12 - 23 - 54 - 89 -
PS D:\109082530025_MOH-Chandra-Wardana_>
```

4) (Judul Guided4)

```
1 package main
2
3 import "fmt"
4
5 type mahasiswa struct {
6     nama, nim string
7     IPK      float64
8 }
9
10 func SelectionSortArray(sMHS *[5]mahasiswa) {
11     var idx_min, i, j int
12     for i = 0; i < len(sMHS)-1; i++ { //loop luar
13         idx_min = i
14         for j = i + 1; j < len(sMHS); j++ { //loop dalam mencari nilai ekstrim
15             if sMHS[j].IPK < sMHS[idx_min].IPK { //sorting terkecil ke terbesar
16                 idx_min = j
17             }
18         }
19         if idx_min != i {
20             sMHS[i], sMHS[idx_min] = sMHS[idx_min], sMHS[i]
21         }
22     }
23 }
24
25 func main() {
26     var structMHS [5]mahasiswa
27
28     for i := 0; i < len(structMHS); i++ {
29         fmt.Printf("--- Masukkan Data Mahasiswa Ke-%d ---\n", i)
30         fmt.Print("Nama : ")
31         fmt.Scan(&structMHS[i].nama)
32         fmt.Print("NIM : ")
33         fmt.Scan(&structMHS[i].nim)
34         fmt.Print("IPK : ")
35         fmt.Scan(&structMHS[i].IPK)
36     }
37     fmt.Println()
38
39     fmt.Println("=== SEBELUM SORITNG ===")
40     for i := 0; i < len(structMHS); i++ {
41         fmt.Printf("--- Data Mahasiswa Ke-%d ---\n", i)
42         fmt.Printf("Nama : %s\n", structMHS[i].nama)
43         fmt.Printf("NIM : %s\n", structMHS[i].nim)
44         fmt.Printf("IPK : %.2f\n", structMHS[i].IPK)
45     }
46     fmt.Println("=====")
47     fmt.Println()
48
49     SelectionSortArray(&structMHS)
50     fmt.Println("=== SETELAH SORITNG ===")
51     for i := 0; i < len(structMHS); i++ {
52         fmt.Printf("--- Data Mahasiswa Ke-%d ---\n", i)
53         fmt.Printf("Nama : %s\n", structMHS[i].nama)
54         fmt.Printf("NIM : %s\n", structMHS[i].nim)
55         fmt.Printf("IPK : %.2f\n", structMHS[i].IPK)
56     }
57     fmt.Println("=====")
58     fmt.Println()
59 }
60
```

Penjelasan Code :

Program ini mendemonstrasikan fleksibilitas algoritma Selection Sort dalam mengurutkan data koleksi kompleks berupa array struct mahasiswa yang

berisi objek bermacam atribut seperti nama, nim, dan IPK. Fungsi SelectionSortArray melakukan pemindaian linear dan perbandingan terfokus pada kolom nilai numerik IPK ($sMHS[j].IPK < sMHS[idx_min].IPK$) guna mengurutkan lima data mahasiswa dari raihan IPK terkecil menuju terbesar, di mana proses pemindahan posisi satu paket data mahasiswa dilakukan secara aman menggunakan operasi swap satu baris. Pada blok main, program secara interaktif menuntun pengguna memasukkan identitas lengkap setiap mahasiswa, mencetak rangkuman profil data keseluruhan sebelum proses eksekusi sorting dilakukan, memicu fungsi pengurutan, dan menampilkan kembali daftar riwayat mahasiswa yang kini telah terstruktur rapi berdasarkan urutan prestasi IPK.

Output :

```
PS D:\109082530025_MOH-Chandra-Wardana_> go run "d:\109082530025_MOH-Chandra-Wardana_
\Week 12 - Modul 14\Guided\guided4\guided4.go"
--- Masukkan Data Mahasiswa Ke-0 ---
Nama : Andi
NIM : 7737373
IPK : 3
--- Masukkan Data Mahasiswa Ke-1 ---
Nama : danas
NIM : 3637373
IPK : 3.20
--- Masukkan Data Mahasiswa Ke-2 ---
Nama : dini
NIM : 7364734
IPK : 4.00
--- Masukkan Data Mahasiswa Ke-3 ---
Nama : candraw
NIM : 7348573
IPK : 3.99
--- Masukkan Data Mahasiswa Ke-4 ---
Nama : wardana
NIM : 26353432
IPK : 3.44
```

<pre>=== SEBELUM SORTING === --- Data Mahasiswa Ke-0 --- Nama : Andi NIM : 7737373 IPK : 3.00 --- Data Mahasiswa Ke-1 --- Nama : danas NIM : 3637373 IPK : 3.20 --- Data Mahasiswa Ke-2 --- Nama : dini NIM : 7364734 IPK : 4.00 --- Data Mahasiswa Ke-3 --- Nama : candraw NIM : 7348573 IPK : 3.99 --- Data Mahasiswa Ke-4 --- Nama : wardana NIM : 26353432 IPK : 3.44 =====</pre>	<pre>=== SETELAH SORTING === --- Data Mahasiswa Ke-0 --- Nama : Andi NIM : 7737373 IPK : 3.00 --- Data Mahasiswa Ke-1 --- Nama : danas NIM : 3637373 IPK : 3.20 --- Data Mahasiswa Ke-2 --- Nama : wardana NIM : 26353432 IPK : 3.44 --- Data Mahasiswa Ke-3 --- Nama : candraw NIM : 7348573 IPK : 3.99 --- Data Mahasiswa Ke-4 --- Nama : dini NIM : 7364734 IPK : 4.00 =====</pre>
---	---

C. Unguided

1. Soal Unguided 1

[Rumah Kerabat Hercules]

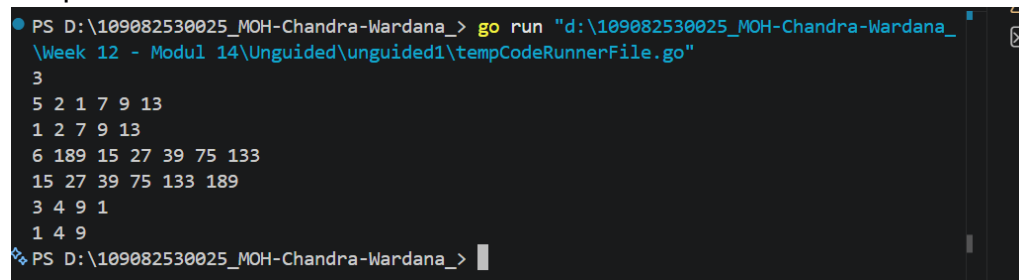
Jawaban :

```
1 package main
2
3 import (
4     "fmt"
5 )
6
7 func selectionSortAscending(arr []int, n int) {
8     for i := 1; i <= n-1; i++ {
9         idxMin := i - 1
10        for j := i; j < n; j++ {
11            if arr[idxMin] > arr[j] {
12                idxMin = j
13            }
14        }
15        t := arr[idxMin]
16        arr[idxMin] = arr[i-1]
17        arr[i-1] = t
18    }
19 }
20
21 func main() {
22     var n, m int
23     if _, err := fmt.Scan(&n); err != nil {
24         return
25     }
26
27     for k := 0; k < n; k++ {
28         fmt.Scan(&m)
29         rumah := make([]int, m)
30         for i := 0; i < m; i++ {
31             fmt.Scan(&rumah[i])
32         }
33
34         selectionSortAscending(rumah, m)
35
36         for i := 0; i < m; i++ {
37             fmt.Printf("%d", rumah[i])
38             if i < m-1 {
39                 fmt.Print(" ")
40             }
41         }
42         fmt.Println()
43     }
44 }
```

Penjelasan Code :

Program ini dibuat untuk mengurutkan nomor rumah kerabat Hercules di beberapa daerah secara membesar menggunakan algoritma Selection Sort sesuai dengan panduan notasi di dalam modul. Fungsi `selectionSortAscending` bekerja dengan mencari indeks elemen terkecil (`idxMin`) di dalam rentang data yang tersisa, lalu menukarkannya dengan elemen di posisi paling kiri dari rentang tersebut. Pada fungsi utama (`main`), program pertama-tama membaca jumlah daerah `n`, lalu melakukan perulangan sebanyak `n` kali untuk membaca jumlah rumah `m` beserta seluruh nomor rumah di daerah tersebut. Setelah data dimasukkan ke dalam slice dinamis, algoritma seleksi dipanggil untuk menyusun data tersebut secara ascending, dan hasilnya langsung dicetak dalam satu baris untuk setiap daerah.

Output :



```
PS D:\109082530025_MOH-Chandra-Wardana> go run "d:\109082530025_MOH-Chandra-Wardana_\Week 12 - Modul 14\Unguided\unguided1\tempCodeRunnerFile.go"
3
5 2 1 7 9 13
1 2 7 9 13
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 4 9
PS D:\109082530025_MOH-Chandra-Wardana>
```

Penjelasan :

Output yang dihasilkan oleh program terdiri dari baris-baris berisi rangkaian nomor rumah kerabat yang telah tersusun rapi dari nilai terkecil hingga terbesar di setiap daerahnya. Sebagai contoh, jika input daerah pertama memiliki data acak berupa nomor rumah seperti "5 2 1 7 9 13", maka program akan memproses perbandingan dan pertukaran nilai ekstrim sehingga menghasilkan baris output terurut yaitu "1 2 7 9 13". Setiap baris keluaran secara akurat memisahkan nomor rumah dengan spasi dan langsung berpindah ke baris baru untuk menampilkan hasil pengurutan daerah berikutnya sesuai dengan data masukan yang diberikan.

2. Soal Unguided 2
Kerabat Dekat Jalan Ganjil & Genap
Jawaban :

```

1 package main
2
3 import (
4     "fmt"
5 )
6
7 func selectionSortAscending(arr []int, n int) {
8     for i := 1; i <= n-1; i++ {
9         idxMin := i - 1
10        for j := i; j < n; j++ {
11            if arr[idxMin] > arr[j] {
12                idxMin = j
13            }
14        }
15        t := arr[idxMin]
16        arr[idxMin] = arr[i-1]
17        arr[i-1] = t
18    }
19 }
20
21 func selectionSortDescending(arr []int, n int) {
22     for i := 1; i <= n-1; i++ {
23         idxMax := i - 1
24        for j := i; j < n; j++ {
25            if arr[idxMax] < arr[j] {
26                idxMax = j
27            }
28        }
29        t := arr[idxMax]
30        arr[idxMax] = arr[i-1]
31        arr[i-1] = t
32    }
33 }
34
35 func main() {
36     var n, m, nomor int
37     if _, err := fmt.Scan(&n); err != nil {
38         return
39     }
40
41     for k := 0; k < n; k++ {
42         fmt.Scan(&m)
43         var ganjil []int
44         var genap []int
45
46         for i := 0; i < m; i++ {
47             fmt.Scan(&nomor)
48             if nomor%2 != 0 {
49                 ganjil = append(ganjil, nomor)
50             } else {
51                 genap = append(genap, nomor)
52             }
53         }
54
55         selectionSortAscending(ganjil, len(ganjil))
56         selectionSortDescending(genap, len(genap))
57
58         var hasil []string
59         for _, v := range ganjil {
60             hasil = append(hasil, fmt.Sprintf("%d", v))
61         }
62         for _, v := range genap {
63             hasil = append(hasil, fmt.Sprintf("%d", v))
64         }
65
66         for i, v := range hasil {
67             fmt.Print(v)
68             if i < len(hasil)-1 {
69                 fmt.Print(" ")
70             }
71         }
72         fmt.Println()
73     }
74 }

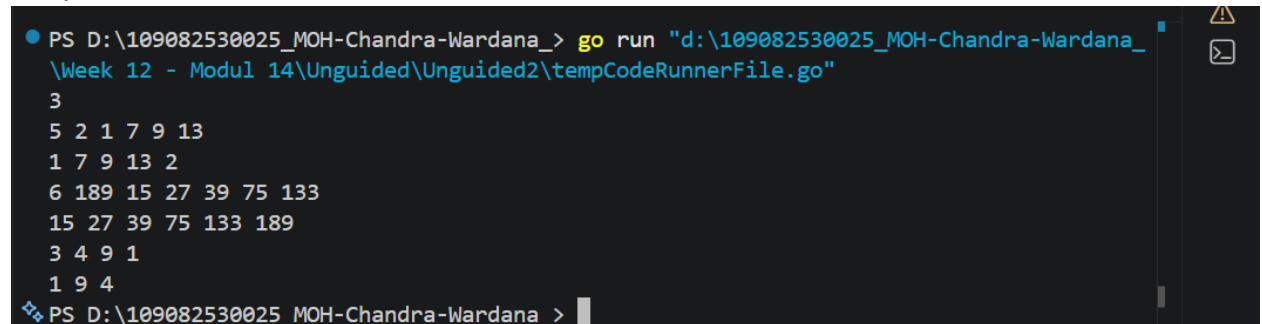
```

Penjelasan Code Program :

Program ini mengimplementasikan petunjuk pertama dari modul, di mana nomor rumah ganjil dan genap dipisahkan ke dalam dua array/slice yang berbeda saat proses membaca data masukan berlangsung. Sesuai dengan

instruksi soal, program ini menggunakan dua variasi fungsi Selection Sort: fungsi selectionSortAscending untuk mengurutkan kelompok nomor ganjil dari yang terkecil, dan fungsi selectionSortDescending yang telah dimodifikasi kondisi perbandingannya untuk mengurutkan kelompok nomor genap dari yang terbesar. Di dalam blok perulangan utama, setiap nomor rumah diperiksa menggunakan operasi modulus (%2), kemudian dimasukkan ke kelompoknya masing-masing sebelum akhirnya kedua kelompok tersebut diurutkan secara independen dan dicetak berurutan dalam satu baris.

Output :



```
PS D:\109082530025_MOH-Chandra-Wardana_> go run "d:\109082530025_MOH-Chandra-Wardana_
\Week 12 - Modul 14\Unguided\Unguided2\tempCodeRunnerFile.go"
3
5 2 1 7 9 13
1 7 9 13 2
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 9 4
PS D:\109082530025_MOH-Chandra-Wardana_>
```

Penjelasan :

Hasil keluaran dari program ini berupa baris data terformat yang mendahulukan semua nomor rumah ganjil dalam kondisi urutan membesar, lalu langsung diikuti oleh semua nomor rumah genap dalam kondisi urutan mengecil. Berdasarkan visualisasi data pada contoh kasus, jika sebuah baris input mengandung kombinasi angka acak seperti "12 1 13 8 2", program berhasil memisahkan dan menyusun nomor ganjil menjadi "1 13" serta nomor genap menjadi "12 8 2", sehingga total output yang tercetak di layar menjadi "1 13 12 8 2". Format keluaran ini secara konsisten menyajikan nomor ganjil dan genap dalam satu rangkaian terpadu yang dipisahkan spasi untuk setiap daerah kerabat Hercules.

3. Soal Unguided 3

Program Median dengan Insertion Sort

Jawaban :

```
1 package main
2
3 import (
4     "fmt"
5 )
6
7 func insertionSortAscending(arr []int, n int) {
8     for i := 1; i <= n-1; i++ {
9         j := i
10        temp := arr[j]
11        for j > 0 && temp < arr[j-1] {
12            arr[j] = arr[j-1]
13            j = j - 1
14        }
15        arr[j] = temp
16    }
17 }
18
19 func main() {
20     var data []int
21     var input int
22
23     for {
24         fmt.Scan(&input)
25         if input == -5313 {
26             break
27         }
28
29         if input == 0 {
30             n := len(data)
31             if n > 0 {
32                 insertionSortAscending(data, n)
33                 var median int
34                 if n%2 != 0 {
35                     median = data[n/2]
36                 } else {
37                     median = (data[(n/2)-1] + data[n/2]) / 2
38                 }
39                 fmt.Println(median)
40             }
41         } else {
42             data = append(data, input)
43         }
44     }
45 }
```

Penjelasan Code :

Program ini dirancang untuk menghitung nilai median dari kumpulan data dinamis yang terus bertambah dengan memanfaatkan algoritma Insertion Sort sesuai instruksi petunjuk soal. Alur program berjalan di dalam perulangan tak terbatas yang akan berhenti membaca data hanya jika mendeteksi nilai penanda akhir yaitu -5313. Setiap kali program menerima input angka 0, data non-nol yang telah tersimpan di dalam slice akan

diurutkan secara membesar (ascending) menggunakan fungsi `insertionSortAscending`, lalu program menghitung posisi nilai tengahnya. Jika jumlah data ganjil, median diambil langsung dari elemen tengah, sedangkan jika genap, median didapat dari rata-rata dua elemen tengah yang dibulatkan ke bawah menggunakan pembagian bilangan bulat di Go.

Output :

```
PS D:\109082530025_MOH-Chandra-Wardana> go run "d:\109082530025_MOH-Chandra-Wardana_\Week 12 - Modul 14\Unguided\unguided3\tempCodeRunnerFile.go"
7 23 11 0 5 19 2 29 3 13 17 0 -5313
11
12
PS D:\109082530025_MOH-Chandra-Wardana>
```

Penjelasan :

Keluaran program adalah nilai median dari seluruh rangkaian data yang telah dibaca hingga saat angka 0 itu ditemukan, di mana nilai tersebut dicetak satu per baris. Sesuai contoh simulasi modul, ketika urutan input awal bernilai "7 23 11 0", program mengurutkan data menjadi "7 11 23" dan mengeluarkan median "11" karena jumlah datanya ganjil. Selanjutnya, saat rentang input berlanjut dan menemukan angka 0 kedua dengan total sepuluh data terurut "2 3 5 7 11 13 17 19 23 29", program secara tepat menghitung rata-rata dua nilai tengah yaitu $(11 + 13) / 2$ sehingga menghasilkan output "12" pada baris berikutnya.

4) Soal Unguided 3

Array Berjarak Sama

Jawaban :

```

1 package main
2
3 import (
4     "fmt"
5 )
6
7 func insertionSortAscending(arr []int, n int) {
8     for i := 1; i < n-1; i++ {
9         j := i
10        temp := arr[i]
11        for j > 0 && temp < arr[j-1] {
12            arr[j] = arr[j-1]
13            j = j - 1
14        }
15        arr[j] = temp
16    }
17 }
18
19 func main() {
20     var input int
21     var data []int
22
23     for {
24         fmt.Scan(&input)
25         if input < 0 {
26             break
27         }
28         data = append(data, input)
29     }
30
31     n := len(data)
32     if n == 0 {
33         return
34     }
35
36     insertionSortAscending(data, n)
37
38     for i := 0; i < n; i++ {
39         fmt.Printf("%d ", data[i])
40         if i < n-1 {
41             fmt.Print(" ")
42         }
43     }
44     fmt.Println()
45
46     if n == 1 {
47         fmt.Println("Data berjarak 0")
48         return
49     }
50
51     jarakAwal := data[1] - data[0]
52     tetap := true
53
54     for i := 1; i < n-1; i++ {
55         if (data[i+1] - data[i]) != jarakAwal {
56             tetap = false
57             break
58         }
59     }
60
61     if tetap {
62         fmt.Printf("Data berjarak %d\n", jarakAwal)
63     } else {
64         fmt.Println("Data berjarak tidak tetap")
65     }
66 }

```

Penjelasan Code :

Program ini berfungsi untuk membaca sekumpulan bilangan bulat non-negatif, mengurutkannya dengan Insertion Sort, dan memeriksa konsistensi selisih (jarak) antar elemen yang berdekatan. Proses input menggunakan perulangan yang akan berhenti seketika saat membaca bilangan negatif, di mana bilangan negatif tersebut diabaikan dan tidak dimasukkan ke dalam array. Setelah elemen diurutkan secara membesar melalui fungsi `insertionSortAscending`, program mencetak seluruh elemen tersebut dalam satu baris. Langkah terakhir adalah melakukan pengecekan linear untuk membandingkan selisih setiap pasangan data berurutan dengan selisih pasangan pertama (`jarakAwal`) guna menentukan status jaraknya.

Output :

```

PS D:\109082530025_MOH-Chandra-Wardana> go run "d:\109082530025_MOH-Chandra-Wardana_
\Week 12 - Modul 14\Unguided\unguided4\tempCodeRunnerFile.go"
43 25 1 19 37 7 31 13 -1
1 7 13 19 25 31 37 43
Data berjarak 6
PS D:\109082530025_MOH-Chandra-Wardana>

```



```

PS D:\109082530025_MOH-Chandra-Wardana_> go run "d:\109082530025_MOH-Chandra-Wardana_
\Week 12 - Modul 14\Unguided\unguided4\tempCodeRunnerFile.go"
4 40 14 8 26 1 38 2 32 -31
1 2 4 8 14 26 32 38 40
Data berjarak tidak tetap
PS D:\109082530025_MOH-Chandra-Wardana_>

```

Penjelasan Output :

Keluaran dari program ini terdiri dari tepat dua baris, di mana baris pertama menampilkan deretan angka yang telah terurut membesar, sedangkan baris kedua menampilkan status hubungan antar angka tersebut. Jika program menerima input acak yang setelah diurutkan membentuk pola aritmatika konstan seperti "1 7 13 19 25 31 37 43", maka baris kedua akan mengonfirmasi dengan teks "Data berjarak 6" karena semua elemennya memiliki selisih yang sama yaitu enam. Sebaliknya, apabila input acak tersebut menghasilkan urutan dengan selisih yang bervariasi seperti "1 2 4 8 14 26 32 38 40", maka program otomatis mencetak kesimpulan status berupa kalimat "Data berjarak tidak tetap".

D. Kesimpulan

Berdasarkan dokumen praktikum yang diberikan, dapat disimpulkan bahwa Modul 14 berfokus pada pemahaman dan implementasi dua algoritma pengurutan linier dasar dalam bahasa Go, yaitu Selection Sort yang bekerja dengan cara memindai dan mencari nilai ekstrim (terkecil/terbesar) untuk diletakkan pada posisi yang seharusnya, serta Insertion Sort yang mengurutkan data dengan cara menyisipkan suatu nilai target langsung ke posisi yang tepat melalui mekanisme pencarian sekuensial dan pergeseran elemen. Kedua metode ini tidak hanya diterapkan pada susunan data primitif seperti bilangan bulat (integer) maupun teks (string), melainkan juga diimplementasikan secara fleksibel untuk memanipulasi struktur data yang lebih kompleks berupa koleksi data bertipe struct. Selain itu, modul ini melatih kemampuan analisis logika pemrograman melalui berbagai studi kasus nyata, seperti pemisahan urutan bilangan ganjil-genap, penentuan nilai median secara dinamis, analisis konsistensi jarak antar-elemen numerik, hingga pengelolaan serta penyaringan data buku berbasis peringkat (rating) di dalam sistem perpustakaan.

